

AP05

LISTES ET TUPLES

I. Les listes

Définition 1 : Les listes

Une **liste** est une structure de donnée qui contient des éléments ou items identifiés par des indices.

Ces éléments sont :

- **ordonnés** : les éléments d'une liste sont rangés dans un ordre précis. Chaque élément a une position dans la liste, appelée indice.
- **séparés par des virgules** : les éléments d'une liste sont séparés par des virgules.
- **entre crochets** : les éléments d'une liste sont entourés de crochets [] en python.

Indice i	0	1	2	3	4	5	6	7
Élément	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]

Créer une liste

</> Code Python

```
1 ma_liste = [1, 2, 3, 4, 5]
```

</> Code Python

```
1 ma_liste = ['a', 'b', 'c']
```

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(type(ma_liste))           # Une liste est de type 'list'
<class 'list'>
```

Accéder aux éléments d'une liste

Pour accéder à un élément d'une liste, on utilise son indice. Les indices commencent à 0.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(ma_liste[0])
1
>>> print(ma_liste[2])
3
```

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(ma_liste[-1])
5
>>> print(ma_liste[-2])
4
```

Modifier les éléments d'une liste

Les éléments d'une liste peuvent être modifiés en utilisant leur indice.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste[0] = 10
>>> print(ma_liste)
[10, 2, 3, 4, 5]
```

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste[4] = 20
>>> print(ma_liste)
[1, 2, 3, 4, 20]
```

Connaitre le nombre d'éléments d'une liste

Pour connaître le nombre d'éléments d'une liste, on utilise la fonction `len()`.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> print(len(ma_liste))
5
```

```
>>> ma_liste = ['a', 'b', 'c']
>>> print(len(ma_liste))
3
```

🔧 Exercice 1 : Manipulation de base

1. Créer la liste suivante : [4, 8, 7, 9, 4]
2. Afficher le nombre d'éléments de cette liste. (*affiche 5*)
3. Afficher le troisième élément de cette liste. (*affiche 7*)
4. Modifier le deuxième élément de cette liste pour qu'il soit égal à 10. (*affiche [4, 10, 7, 9, 4]*)
5. Modifier le dernier élément de cette liste pour qu'il soit égal à 5. (*affiche [4, 10, 7, 9, 5]*)
6. Modifier le premier éléments de la liste en le remplaçant par la somme du premier et du troisième élément de cette liste. (*affiche [11, 10, 7, 9, 5]*)

Itérer sur une liste avec une boucle FOR

Comme pour les chaînes de caractères, On peut itérer sur les éléments d'une liste à l'aide d'une boucle FOR.

</> Code Python

```
1 ma_liste = [4, 8, 7, 9, 4]
2 for i in range(0, len(ma_liste)):
3     print(ma_liste[i])
```

</> Code Python

```
1 ma_liste = [4, 8, 7, 9, 4]
2 for elt in ma_liste:
3     print(elt)
```

🔧 Exercice 2 : Quelques algorithmes sur les listes

1. Ecrire une fonction `somme(liste)` qui renvoie la somme des éléments d'une liste de nombres.

</> Code Python

```
def somme(liste):
    ''' Renvoie la somme des éléments de liste'''



    assert somme([1, 2, 3, 4, 5]) == 15
    assert somme([4, 8, 7, 9, 4]) == 32
```

2. Ecrire une fonction `maximum(liste)` qui renvoie le maximum des éléments d'une liste de nombres.

</> Code Python

```
def maximum(liste):
    ''' Renvoie le maximum des éléments de liste'''



    assert maximum([1, 2, 3, 4, 5]) == 5
    assert maximum([4, 8, 7, 9, 4]) == 9
```

3. Ecrire une fonction `est_present(elt, liste)` qui renvoie `True` si l'élément `elt` est présent dans la liste `liste`, et `False` sinon.

</> Code Python

```
def est_present(elt, liste):
    ''' Renvoie True si elt est dans liste et False sinon '''

    

    assert est_present(3, [1, 2, 3, 4, 5]) == True
    assert est_present(6, [1, 2, 3, 4, 5]) == False
```

Liste de listes

Une liste peut contenir des éléments qui sont eux-mêmes des listes. On parle alors de liste de listes.

```
>>> ma_liste = [[1, 2, 3], [4, 5], [7, 8, 9, 10]]
>>> print(len(ma_liste))
3
>>> print(len(ma_liste[1]))
2
>>> print(ma_liste[0])
[1, 2, 3]
>>> print(ma_liste[1])
[4, 5]
>>> print(ma_liste[2])
[7, 8, 9, 10]
>>> print(ma_liste[0][1])
2
>>> print(ma_liste[1][1])
5
>>> print(ma_liste[2][0])
7
```

🔧 Exercice 3 : Manipulation de listes de listes

1. Créer la liste suivante : `[[1, 2], [3, 4], [5, 6]]`
2. Afficher le nombre d'éléments de cette liste. (*affiche 3*)
3. Afficher le deuxième élément de cette liste. (*affiche [3, 4]*)
4. Afficher le premier élément du troisième élément de cette liste. (*affiche 5*)
5. Modifier le deuxième élément du premier élément de cette liste pour qu'il soit égal à 10
6. Afficher la liste modifiée. (*affiche [[1, 10], [3, 4], [5, 6]]*)
7. Modifier le premier élément du deuxième élément de cette liste pour qu'il soit égal à 20
8. Afficher la liste modifiée. (*affiche [[1, 10], [20, 4], [5, 6]]*)
9. Modifier le deuxième élément du troisième élément de cette liste en le remplaçant par la somme du premier et du deuxième élément de ce troisième élément
10. Afficher la liste modifiée. (*affiche [[1, 10], [20, 4], [5, 11]]*)

Exercice 4 : Itérer sur une liste de listes

1. Ecrire une fonction `somme_liste_de_listes(liste)` qui renvoie la somme des éléments d'une liste de listes de nombres.

 Code Python

```
def somme_liste_de_listes(liste):  
    ''' Renvoie la somme des éléments de liste'''
```



```
assert somme_liste_de_listes([[1, 2], [3, 4], [5, 6]]) == 21
```

2. Ecrire une fonction `maximum_sous_liste(liste)` qui renvoie la sous liste dont la somme des éléments est maximale.

 Code Python

```
def maximum_sous_liste(liste):  
    ''' Renvoie la sous liste dont la somme des éléments est max'''
```



```
assert maximum_sous_liste([[3, 4, 16], [5, 6, 1, 8]]) == [3, 4, 16]
```

Quelques méthodes sur les listes

- `append(elt)` : ajoute l'élément `elt` à la fin de la liste
- `pop()` : supprime et renvoie le dernier élément de la liste
- `pop(i)` : supprime et renvoie l'élément d'indice `i` de la liste

```
>>> ma_liste = [1, 2, 3, 4, 5]  
>>> ma_liste.append(6)  
>>> print(ma_liste)  
[1, 2, 3, 4, 5, 6]  
>>> a = ma_liste.pop()  
>>> print(a)  
6  
>>> print(ma_liste)  
[1, 2, 3, 4, 5]  
>>> b = ma_liste.pop(2)  
>>> print(b)  
3  
>>> print(ma_liste)  
[1, 2, 4, 5]
```

Créer une liste par itération

</> Code Python

```
1 ma_liste = []
2 for i in range(0, 5):
3     ma_liste.append(i)
4 print(ma_liste)      # [0, 1, 2, 3, 4]
```

</> Code Python

```
1 ma_liste = []
2 for i in range(0, 5):
3     ma_liste = [i] + ma_liste
4 print(ma_liste)     # [4, 3, 2, 1, 0]
```

Créer une liste par compréhension

</> Code Python

```
1 ma_liste = [i for i in range(0, 5)]
2 print(ma_liste)      # Affiche: [0, 1, 2, 3, 4]
```

Exercice 5 : Quelques méthodes sur les listes

1. Ecrire une fonction `puissance_2(n)` qui renvoie une liste contenant les puissances de 2 de 0 à n inclus.

</> Code Python

```
def puissance_2(n):
    ''' Renvoie une liste contenant les puissances de 2 de 0 à n '''

    #

    assert puissance_2(3) == [1, 2, 4, 8]
    assert puissance_2(5) == [1, 2, 4, 8, 16, 32]
```

2. Ecrire une fonction `est_pair(liste)` qui renvoie une liste contenant les éléments de liste qui sont pairs.

</> Code Python

```
def est_pair(liste):
    ''' Renvoie une liste contenant les éléments pairs'''

    #

    assert est_pair([1, 2, 3, 4, 5]) == [2, 4]
    assert est_pair([1, 3, 5, 7, 9]) == []
    assert est_pair([2, 4, 6, 21, 8, 12, 13, 10]) == [2, 4, 6, 8, 12, 10]
```

3. Ecrire une fonction `supprime_element(liste, element)` qui supprime les éléments d'une liste en partant du dernier jusqu'à l'obtention du premier 0.

</> Code Python

```
def supprimer_element(liste):  
    ''' Supprime les éléments de liste en partant du dernier  
        jusqu'à l'obtention du premier 0 '''  
  
    #  
    assert supprimer_element([0, 0, 1, 1, 0, 1, 1, 1]) == [0, 0, 1, 1, 0]  
    assert supprimer_element([1, 1, 0, 1, 1, 1, 1, 1, 1]) == [1, 1, 0]
```

4. Ecrire une fonction `est_triee(liste)` qui renvoie True si la liste `liste` est triée dans l'ordre croissant, et False sinon.

</> Code Python

```
def est_triee(liste):  
    ''' Renvoie True si liste est triée dans l'ordre croissant et False sinon '''  
  
    #  
  
    assert est_triee([1, 2, 3, 4, 5]) == True  
    assert est_triee([5, 4, 3, 2, 1]) == False
```

Le hasard et les listes

- La fonction `random.choice(liste)` permet de choisir un élément au hasard dans une liste.
- La fonction `random.shuffle(liste)` permet de mélanger les éléments d'une liste de manière aléatoire.

```
>>> import random  
>>> ma_liste = [1, 2, 3, 4, 5]  
>>> print(random.choice(ma_liste))  
4  
>>> print(ma_liste)  
[1, 2, 3, 4, 5]  
>>> random.shuffle(ma_liste)  
>>> print(ma_liste)  
[1, 5, 3, 4, 2]
```

Le slicing

Le slicing permet de créer une nouvelle liste à partir d'une liste existante en sélectionnant une portion de cette liste.

</> Code Python

```
1 ma_liste = [1, 2, 3, 4, 5]
2 print(ma_liste[1:4])
3 # Affiche: [2, 3, 4]
4 print(ma_liste[:3])
5 # Affiche: [1, 2, 3]
6 print(ma_liste[2:])
7 # Affiche: [3, 4, 5]
```



Les effets de bord sur les listes

- Les listes sont des objets mutables, c'est à dire que leur contenu peut être modifié après leur création.
- Lorsque l'on affecte une liste à une variable, on crée une référence à cette liste. Si on modifie la liste via cette variable, la modification sera visible via toutes les variables qui font référence à cette liste.

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste2 = ma_liste
>>> ma_liste2[0] = 10
>>> print(ma_liste)
[10, 2, 3, 4, 5]
>>> print(ma_liste2)
[10, 2, 3, 4, 5]
```

Si on veut faire une vraie copie d'une liste, on peut utiliser la syntaxe suivante :

```
>>> ma_liste = [1, 2, 3, 4, 5]
>>> ma_liste2 = ma_liste[:]
>>> ma_liste2[0] = 10
>>> print(ma_liste)
[1, 2, 3, 4, 5]
>>> print(ma_liste2)
[10, 2, 3, 4, 5]
```

Lorsqu'on utilise des listes dans des fonctions, il faut faire attention aux effets de bord. Si on modifie une liste passée en argument d'une fonction, la modification sera visible à l'extérieur de la fonction.

Dans cet exemple, il n'est pas utile de renvoyer la liste modifiée, car la modification est déjà visible à l'extérieur de la fonction.

</> Code Python

```
1 def ajoute_element(liste, element):
2     liste.append(element)
3
4 ma_liste = [1, 2, 3, 4, 5]
5 ajoute_element(ma_liste, 6)
6 print(ma_liste)           # Affiche: [1, 2, 3, 4, 5, 6]
```



🔗 Exercice 6 : Effets de bord et fonctions

Ecrire une fonction `decalage_gauche(liste)` qui décale les éléments de la liste `liste` vers la gauche, en mettant le premier élément à la fin de la liste. La liste doit être modifiée en place.

🔗 Code Python

```
def decalage_gauche(liste):  
    ''' Décale les éléments de liste vers la gauche '''  
  
  
  
assert decalage_gauche([1, 2, 3, 4, 5]) == [2, 3, 4, 5, 1]  
assert decalage_gauche([5, 4, 3, 2, 1]) == [4, 3, 2, 1, 5]
```

🔗 Exercice 7 : Les listes

Ecrire une fonction qui renvoie une nouvelle liste contenant la somme des éléments de deux listes de même longueur. La fonction doit vérifier que les deux listes ont la même longueur. Si les deux listes ne sont pas de même longueur, la liste la plus courte est complétée par des zéros jusqu'à ce qu'elle ait la même longueur que l'autre liste.

🔗 Code Python

```
def somme_listes(L1, L2):  
    ''' Renvoie une nouvelle liste contenant la somme des éléments de L1 et L2 '''  
  
  
  
assert somme_listes([1, 2, 3], [4, 5, 6]) == [5, 7, 9]  
assert somme_listes([1, 2], [3, 4, 5]) == [4, 6, 5]
```

Exercice 8 : Les listes

Écrire une fonction qui modifie une liste en remplaçant chaque élément par la somme de tous les éléments qui sont à sa droite. Par exemple, si la liste est [1, 2, 3, 4], la liste modifiée doit être [10, 9, 7, 4]. La fonction doit modifier la liste en place.

</> Code Python

```
def somme_droite(liste):
    ''' Modifie la liste en remplaçant chaque élément par la somme de tous
        les éléments à sa droite'''

    assert somme_droite([1, 2, 3, 4]) == [10, 9, 7, 4]
    assert somme_droite([5, 4, 3, 2, 1]) == [15, 10, 6, 3, 1]
```



II. Les tuples

Définition 1 : Les tuples

Un **tuple** est une structure de donnée qui contient des éléments ou items identifiés par des indices. Les éléments d'un tuple sont :

- **ordonnés** : chaque élément a une position dans le tuple, appelée indice.
- **séparés par des virgules** : les éléments d'un tuple sont séparés par des virgules.
- **entre parenthèses** : les éléments d'un tuple sont entourés de parenthèses () en python.

Indice i	0	1	2	3	4	5	6	7
Élément	T[0]	T[1]	T[2]	T[3]	T[4]	T[5]	T[6]	T[7]

Créer un tuple

</> Code Python

```
1 mon_tuple = (1, 2, 3, 4, 5)
```

</> Code Python

```
1 mon_tuple = ('a', 'b', 'c')
```

```
>>> mon_tuple = (1, 2, 3, 4, 5)
>>> print(type(mon_tuple))      # Un tuple est de type 'tuple'
<class 'tuple'>
```

Propriété 1 : Immutabilité des tuples

Les éléments d'un tuple ne peuvent pas être modifiés après la création du tuple. Un tuple est une structure de donnée immuable.